

Lab 02 — Images, lagen en registries

Theorie — hoe een image aan haar naam komt, waaruit ze bestaat, waar ze vandaan komt, en waarom Podman weigert te gokken.

Doelstellingen

- Een volledige imagenaam ontleden en weten wat Docker stilzwijgend aanvult — en wat Podman weigert aan te vullen.
- Begrijpen waarom een **tag** een verplaatsbaar label is en een **digest** een identiteit.
- Het **lagenmodel** uitleggen en wat het betekent voor schijfruimte en netwerkverkeer.
- Een image inspecteren zonder ze te starten.
- Weten waar Docker Hub, private registries en "officiële" images thuishoren.

1. De volledige naam van een image

Jij typt `podman pull postgres:16-alpine`. De engine leest daarin:

```
docker.io / library / postgres : 16-alpine
└registry┐ └namespace┐ └repo┐ └tag┐
```

Deel	Rol	Standaardwaarde
registry	De server die de image host	<code>docker.io</code> (Docker Hub)
namespace	De organisatie of gebruiker die eigenaar is	<code>library</code> (officiële images)
repository	De naam van de applicatie	<i>verplicht</i>
tag	De versie	<code>latest</code>

Daaruit volgen meteen drie dingen:

- `postgres` alleen betekent `docker.io/library/postgres:latest`.
- Een bedrijfsimage draagt een volledige naam, bijvoorbeeld `registry.mijnbedrijf.be/team-betalingen/api-facturatie:1.4.2`. Staat er een **punt** of een **poort** in het eerste deel, dan gaat het om een registry en niet om een namespace.
- De images in de namespace `library` zijn de **officiële images**: onderhouden samen met Docker, geauditeerd en gedocumenteerd (`postgres`, `nginx`, `node`, `eclipse-temurin`). Een image als `bobvan59/postgres` biedt geen van die garanties.

PODMAN

Docker maakt van `postgres` stilzwijgend `docker.io/library/postgres`. Podman vindt dat gevaarlijk: een korte naam kan naar een heel andere image verwijzen naargelang de registry die bevraagd wordt (*typosquatting*, of een naamgenoot op een interne registry). Daarom volgt Podman `registries.conf`: gekende **aliassen** (`alpine`, `nginx`, `postgres` ...) lost het op zonder vragen, en voor de rest raadpleegt het de lijst `unqualified-search-registries` — staan daar meerdere registries in, dan moet je kiezen. Op Ubuntu bevat die lijst alleen `docker.io`, dus korte namen "werken gewoon"; op Fedora of een `podman machine` krijg je een vraag. Een image die je lokaal bouwt, krijgt het voorvoegsel `localhost/`: `podman build -t api:1.0` levert `localhost/api:1.0` op. `podman images` toont altijd de volledige naam — geen giswerk.

VALKUIL

`latest` betekent niet "de nieuwste versie". Het is gewoon de **standaard**-tag; de uitgever verplaatst hem, of net niet, en hij kan rustig naar een build van twee jaar oud wijzen. In productie is `latest` uit den boze: je uitrol is niet meer reproduceerbaar en er valt niets terug te draaien.

2. Verplaatsbare tag, onveranderlijke digest

Een **tag** is een pointer. `postgres:16` wijst vandaag naar `16.10` en morgen naar `16.11`; op je schijf verandert er niets, maar de volgende `pull` haalt iets anders binnen. Een **digest** is de SHA-256-vingerafdruk van het manifest: `postgres@sha256:9d0d1f1e...`. Omdat hij **uit de inhoud berekend wordt**:

- zijn twee images met dezelfde digest bit voor bit identiek, waar ze ook staan;
- kan een image niet veranderen zonder dat de digest mee verandert;
- kun je een uitrol dus exact vastpinnen.

→ BEVEILIGING

SHA-256 is een **hashfunctie**: ze zet eender welke invoer om in 64 hexadecimale tekens, deterministisch (zelfde invoer, zelfde uitvoer) en in één richting (niemand kan een invoer maken die een gekozen uitvoer oplevert). Wijzig één bit in de image en de hele vingerafdruk verandert. Git identificeert commits volgens hetzelfde principe: het identificatienummer is meteen ook een integriteitsbewijs.

```
podman image inspect --format '{{.Digest}}' postgres:16-alpine
podman pull docker.io/library/postgres@sha256:9d0d1f1e... # perfect reproduceerbaar
```

De meeste bedrijven komen bij hetzelfde compromis uit: één unieke, onveranderlijke tag per build (`api:1.4.2` of `api:2026.03.17-b318`) die nooit hergebruikt wordt, plus uitroltools die de digest vastpinnen.

3. Een image is een stapel lagen

Elke bouw instructie die het bestandssysteem wijzigt, produceert een **laag** (*layer*): de bestanden die toegevoegd, gewijzigd of verwijderd zijn ten opzichte van de vorige toestand. De uiteindelijke image is niets meer dan die lagen op elkaar, plus een manifest dat ze oplijst en een configuratie (standaardcommando, variabelen, gebruiker...).

	laag 4: COPY app.jar	(60 MB)
	laag 3: de JRE	(180 MB)
	laag 2: systeempakketten	(30 MB)
	laag 1: basis Debian slim	(75 MB)

↑ alleen-lezen, gedeeld door alle images die ze bevatten

→ LINUX

Het opslagstuurprogramma `overlay` is een kernelbestandssysteem dat mappen op elkaar **stapelt**: alleen-lezen "onderlagen" met daarboven één beschrijfbaar "bovenlaag". Lezen zoekt van boven naar onder. Schrijven kopieert het bestand eerst naar de bovenlaag (*copy-on-write*). Verwijderen maakt een spookbestand (*whiteout*) dat het origineel verbergt zonder het weg te nemen. Al het gedrag van images volgt uit die drie regels.

Die structuur verklaart vier dingen die je voortdurend zult tegenkomen:

- 1. Gedeelde schijfruimte.** Vertrekken je twaalf microservices allemaal van dezelfde JRE, dan staan die 180 MB **één keer** op schijf. De kolom `SIZE` van `podman images` optellen geeft dus een veel te hoog cijfer — `podman system df` toont het echte verbruik.
- 2. Differentiële overdracht.** Een `pull` of `push` verstuurt alleen de lagen die aan de andere kant ontbreken: bij een nieuwe uitrol van je API gaat vaak alleen de JAR-laag over de lijn.
- 3. Onveranderlijkheid, fouten inbegrepen.** Voegt een laag een wachtwoord toe en verwijdert een latere laag het weer, dan **zit dat bestand nog altijd in de image**. De latere laag verbergt het alleen, en wie de image in handen krijgt, kan het terughalen. Een geheim hoort nooit in een build thuis (lab 08).
- 4. De buildcache.** Lagen worden op inhoud geïdentificeerd, dus de engine hergebruikt elke laag die ze al heeft (lab 04).

ONTHOUDEN

Lagen worden gedeeld per host of per registry, niet per image: een identieke laag in twee images staat maar één keer op schijf. In rootless-modus zit die opslag in **jouw** homemap (`~/.local/share/containers/storage`): twee gebruikers op dezelfde machine delen dus niets.

VALKUIL

Een tag verwijst in werkelijkheid naar een **manifest list**, met één ingang per architectuur (`linux/amd64`, `linux/arm64`...); de `pull` kiest die van jouw machine. Daarom weigert een image die op een MacBook met Apple Silicon gebouwd is, te starten op een `amd64`-server: `exec format error`. Met `--platform` dwing je de architectuur af.

4. De dagelijkse commando's

```
podman pull nginx:alpine           # downloaden zonder starten
podman images                      # lokale images ophalen
podman images --filter dangling=true # images zonder tag (weeslagen)
podman history nginx:alpine         # de lagen, hun grootte en herkomst
podman image tree nginx:alpine      # de lagen... als boom, met hun bronimage
podman image inspect nginx:alpine   # volledige metadata als JSON
podman tag nginx:alpine mijn-nginx:v1 # een naam toevoegen (wordt localhost/mijn-nginx:v1)
podman rmi mijn-nginx:v1            # een naam verwijderen (en de image als het de
# laatste was)
podman system df                    # werkelijk gebruikte ruimte
```

Twee subtiliteiten waar veel mensen over struikelen:

podman tag kopieert niets. Het plakt een extra label op dezelfde image; beide namen tonen dezelfde `IMAGE ID`. Omgekeerd geldt hetzelfde: `rmi` op een image met twee tags verwijdert alleen de tag. De data verdwijnt pas wanneer de laatste naam verdwijnt.

Een "dangling" image (`<none>:<none>`) is geen mysterieuze rommel. Haar tag is gewoon doorgeschoven naar een recentere build: de image is haar naam kwijt maar neemt nog schijfruimte in. Dat is het normale restje van herhaalde rebuilds.

Een image uit de engine halen

```
podman save -o api.tar mijn-api:1.0 # archief in docker-archive-formaat
podman save --format oci-archive -o api.tar mijn-api:1.0 # hetzelfde, in het OCI-standaardformaat
podman load -i api.tar # importeert de image opnieuw, tags inbegrepen
```

Dat is handig wanneer de doelmachine niet bij een registry kan (een afgesloten omgeving). Verwar het niet met `export` / `import`: die slaan het bestandssysteem **van een container** plat en gooien zowel de lagen als de configuratie weg (`CMD` , `ENV` , `EXPOSE` ...).

5. De registries

Een registry is een HTTP-dienst die lagen en manifesten opslaat achter een gestandaardiseerde API (`/v2/...`) die elke tool verstaat.

→ HTTP

Een REST-API stelt *resources* beschikbaar via URL's en bewerkt ze met HTTP-werkwoorden: `GET /v2/_catalog` lijst de repositories op, `HEAD /v2/api/manifests/1.0` geeft de digest terug in een header, `PUT` uploadt een laag. Met een simpele `curl` kun je dus met een registry praten — dat doe je straks in het praktijklab. Een Spring Boot-API draait op precies dezelfde mechaniek.

Type	Voorbeelden	Gebruik
Publiek	Docker Hub, <code>ghcr.io</code> , <code>quay.io</code>	Basisimages en kant-en-klare software
Privaat, beheerd	AWS ECR, Azure ACR, Google AR	Eigen images, gehost in de cloud
Privaat, zelf gehost	Harbor, Nexus, GitLab Registry	Eigen images, volledige controle, kwetsbaarheidsscans

Zo verloopt het in een bedrijf:

```
podman login registry.mijnbedrijf.be
podman tag api:1.4.2 registry.mijnbedrijf.be/betalingen/api:1.4.2
podman push registry.mijnbedrijf.be/betalingen/api:1.4.2
```

Drie dingen die je moet weten:

- **podman login bewaart het token** in `${XDG_RUNTIME_DIR}/containers/auth.json` , een tijdelijk bestand dat verdwijnt zodra je je afmeldt. Docker schrijft het daarentegen naar `~/.docker/config.json` — enkel base64-gecodeerd, en het blijft daar voorgoed staan. Op een CI-agent gebruik je beter kortstondige credentials.
- **Podman eist TLS.** Een registry over gewone HTTP — zoals degene die je straks op `localhost:5000` start — wordt geweigerd met `http: server gave HTTP response to HTTPS`

client, tot je `--tls-verify=false` meegeeft of de registry als `insecure = true` opneemt in `registries.conf`. Docker maakt stilzwijgend een uitzondering voor `localhost`; Podman niet.

- **Docker Hub beperkt anonieme downloads** (een quotum per IP-adres). Op een CI zie je dan `toomanyrequests` verschijnen; daarom draaien teams een *pull-through cache* of houden ze een interne kopie van de basisimages bij.

6. In het bedrijf

Op een Spring Boot + Angular-stack:

- De **basisimages** (`eclipse-temurin`, `node`, `nginx`, `postgres`) worden naar de interne registry gekopieerd, vaak met `skopeo copy` — de zuster tool van Podman, die rechtstreeks van registry naar registry kopieert zonder iets te downloaden. In productie haalt niemand iets van het internet: quota, beschikbaarheid en controle over wat binnenkomt pleiten daar allemaal tegen.
- De CI bouwt `registry.intern/mijnapp/api:<versie>` en `.../web:<versie>` en pusht ze; de versie komt van de Git-tag of het buildnummer. Een **scanner** (Trivy, Harbor, Gype) blokkeert images met kritieke kwetsbaarheden. Een uitrol verwijst naar een exacte versie, nooit naar `latest`.

Onthouden

- Een volledige naam luidt `registry/namespace/repository:tag`; de standaardwaarden zijn `docker.io`, `library` en `latest`. Podman toont altijd de volledige naam en zet `localhost/` voor je eigen builds.
- `latest` is niet "de nieuwste versie": het is een standaard-tag, en in productie hoort hij niet thuis.
- Een **tag** kan bewegen; een **digest** `sha256:...` identificeert een exacte inhoud.
- Een image is een stapel **alleen-lezen lagen**, gedeeld tussen images en differentieel overgedragen. Een bestand dat in een latere laag verwijderd is, zit nog altijd in de image — dus nooit een geheim in een build.
- `tag` dupliceert niets; `rmi` verwijdert eerst een naam, geen data. Podman eist TLS: `--tls-verify=false` alleen voor een lokale testregistry.
- `save` / `load` vervoeren een volledige image; `export` / `import` slaan een container plat en verliezen zijn configuratie.

Woordenschat

repository: de versies van een image. — **tag**: verplaatsbaar label. — **digest**: onveranderlijke SHA-256-vingerafdruk. — **manifest**: beschrijving van lagen en configuratie; een **manifest list** indexeert meerdere architecturen. — **dangling image**: image die haar tag kwijt is. — **layer**: laag met bestandswijzigingen. — **overlay**: stuurprogramma dat de lagen stapelt. — **pull-through cache**: lokale spiegel van een publieke registry. — **officiële image**: namespace `library` op Docker Hub. — **short name**: naam zonder registry, opgelost via `registries.conf`. — **skopeo**: images kopiëren en inspecteren tussen registries.